Top 5 Tips for Dataset Preparation

1. **Number of documents**
   You want to be able to demonstrate effective searching, sorting and filtering, and this is best demonstrated when there are sufficient documents in the collection to make these operations useful. **Consider 100 as a minimum size for the main collection**, but many datasets will have much more than this.

2. **Range of field types**
   Different field types lend themselves to different types of queries. For example, text fields are almost always searched on a complete exact match to the search term, but numeric types also lend themselves to range checking (i.e. search for documents containing values greater than/less than/between/etc.). Other field types can also be useful in providing additional functionality, for example URLs, filenames (for images), coordinate data, etc. **You should aim to cover as many field types as possible** – even if it means adding synthetic data to your collection.

3. **Sub-document collections**
   At least one sub-document collection should be seen as an essential element to your dataset. Our definition of a sub-document collection is where the sub-document element is a complete MongoDB document (including `_id` field) and where the parent document can potentially contain multiple sub-document entries. For example, if your top level data element is "movies" and you decide to represent the producer, director and screenplay writer into a separate embedded document called "production", this is NOT a sub-document collection, since any movie will only have at most a single "production" element. On the other hand, "reviews" would be a valid sub-document collection as a single movie will have multiple reviews.

4. **This is not SQL**
   In SQL databases, the aim is to structure the tables (using normalisation) so that all repetition of data is eliminated. Consequently, queries often need to retrieve information from multiple tables (using joins or link tables) in order to generate the required result. Document Store Databases such as MongoDB should be organised to support efficient retrieval, so that a query can be satisfied by accessing a single collection. **Where your dataset consists of multiple files, for example "movies", "reviews", "posters"; you should restructure it to make use of fields and sub-document collections.** For example, the "poster" could be a field of the "movie" entry, while the "reviews" should be a sub-document collection within "movies". In this way, a single query to the "movies" collection should be able to retrieve the movie data, plus (the URL of) the "poster", plus the set of "reviews".

5. **Remember the Front-end**
   Ultimately, you will build a front-end app that consumes your API endpoints. The quality of the app can be greatly enhanced by visual content such as images, charts, graphs, maps, etc. Although such content is not visible in the JSON data returned by your API endpoints, you should **consider how it might be represented visually by the front-end** and make sure that your dataset includes such content.